

■ COMPLETE NOTES - TOPIC 06

CSS Sizing Units

All CSS Units - Basic se Advanced Practical Level tak

Web Development Course - Shauray

px

%

em / rem

vw / vh

vmin / vmax

svh / lvh / dvh

ch / ex / lh

fr

Container Units

calc / min / max / clamp

CSS Units Kya Hote Hain?

Size, distance, spacing aur responsive layout measure karne ka system

SIMPLE DEFINITION

CSS unit ek measurement hoti hai jo browser ko batati hai ki width, height, margin, padding, font-size, gap, border-radius, position, transform ya animation ka value kitna hona chahiye.

CSS mein value usually number + unit hoti hai. Example: `20px`, `2rem`, `50%`, `100vh`. Agar unit galat choose kar di, layout ya to fixed ho jata hai, ya unpredictable ho jata hai. Responsive design ka main secret correct units choose karna hai.

● ● ● basic unit example

```
.card {  
  width: 80%;  
  max-width: 720px;  
  padding: 2rem;  
  font-size: 1rem;  
}
```

■ Unit Categories

Category	Units	Use
Absolute	px, cm, mm, Q, in, pt, pc	Fixed measurement, mostly print or exact UI details
Font Relative	em, rem, ch, ex, cap, ic, lh, rlh	Text-based sizing, spacing, readable layouts
Percentage	%	Parent ke size ke relative sizing
Viewport	vw, vh, vmin, vmax, svw, svh, lvw, lvh, dvw, dvh, vi, vb	Screen / viewport ke relative sizing
Container	cqw, cqh, cqi, cqb, cqmin, cqmax	Parent container ke size ke relative advanced responsive UI
Grid	fr	CSS Grid mein available space divide karna
Other CSS Units	deg, rad, turn, s, ms, dpi, dppx	Rotation, animation timing, image resolution

■ Unitless Values

Har number ke saath unit zaroori nahi hoti. Kuch properties unitless value accept karti hain, jaise `line-height`, `opacity`, `font-weight`, `z-index`, `flex-grow`.

● ● ● unitless values

```
.text {
  line-height: 1.6;      /* unitless recommended */
  opacity: 0.8;
  font-weight: 700;
  z-index: 10;
}
```

■ **Golden Start:** Web layout mein most common units hain `px`, `%`, `rem`, `em`, `vw/vh`, `fr`, aur modern responsive functions `calc()`, `min()`, `max()`, `clamp()`.

Absolute Units

Fixed size units jo parent ya viewport ke hisaab se change nahi hote

DEFINITION

Absolute units ka size fixed hota hai. Inka value parent font-size, screen width, ya container width se automatically change nahi hota. Web design mein sabse common absolute unit `px` hai.

1. px - Pixel

`px` CSS ka most common fixed unit hai. Browser mein 1px ek CSS pixel hota hai, physical device pixel zaroori nahi. High-DPI screens par browser CSS pixel ko internally scale karta hai.

pixel use

```
.box {  
  width: 320px;  
  height: 180px;  
  border-radius: 8px;  
}
```

PX FIXED WIDTH DEMO

80px



160px



240px



Absolute Units Complete Table

Unit	Name	Relationship	Best Use
px	Pixel	1px = 1/96 inch in CSS reference	Border, icon size, exact spacing, UI details
in	Inch	1in = 96px	Print CSS, physical size documents
cm	Centimeter	1cm = 37.8px approx	Print layout
mm	Millimeter	1mm = 3.78px approx	Print layout
Q	Quarter millimeter	1Q = 0.25mm	Very fine print measurements
pt	Point	1pt = 1/72 inch	Print typography
pc	Pica	1pc = 12pt	Print typography

■ Absolute Units Kab Use Karein?

GOOD USE

Thin borders: 1px.
 Icon sizes: 20px, 24px.
 Small radius: 4px, 8px.
 Box shadow blur values.
 Print document measurements.

AVOID

Full page layout fixed px mein.
 Mobile responsive widths fixed px mein.
 Body font-size too small fixed px.
 Card width only 900px without max-width logic.
 Spacing system completely random px values.

■ **Warning:** `px` useful hai, but pure responsive layout ke liye sirf px par depend karna risky hai. Width ke liye often `%`, `max-width`, `rem`, `vw`, ya `clamp()` better hote hain.

em, rem, ch, ex, lh aur rlh

Font ke size aur text metrics ke relative units

1. em - Parent/Current Font Size Ke Relative

em current element ke font-size ke relative hota hai. Agar font-size 20px hai, to **1em = 20px**. Font-size property par **em** parent font-size ke relative calculate hota hai. Padding, margin, width etc. par

em current element ke computed font-size ke relative hota hai.

1em = current element ka font-size

em example

```
.button {  
  font-size: 16px;  
  padding: 0.75em 1.25em; /* 12px 20px */  
}
```

em buttons aur components mein useful hai, kyunki component ka padding font-size ke saath scale ho jata hai.

2. rem - Root Font Size Ke Relative

rem ka full form root em hai. Yeh always **html** element ke font-size ke relative hota hai. Default browser root font-size usually 16px hota hai, so **1rem = 16px** by default.

1rem = html/root font-size

```
html {  
  font-size: 16px;  
}  
  
.card {  
  padding: 2rem; /* 32px */  
  gap: 1rem; /* 16px */  
}
```

■ **Best Practice:** Most projects mein font sizes, spacing scale, layout gaps aur component padding ke liye `rem` clean choice hoti hai, kyunki yeh predictable hoti hai aur user browser settings ko respect karti hai.

■ em vs rem Difference

Unit	Relative To	Best Use	Risk
em	Current/parent font-size	Component internal spacing that scales with text	Nesting se compound size ho sakta hai
rem	Root html font-size	Global font sizes and spacing system	Root font-size change se whole scale change hota hai

■ 3. ch - Character Width Unit

`ch` current font ke "o" character ki width ke relative hota hai. Yeh readable text width ke liye very useful hai. Example: paragraphs ko `60ch` max-width dena reading comfortable banata hai.

```
.article {  
  max-width: 65ch;  
}
```

▪ 4. ex, cap, ic, lh, rlh

Unit	Relative To	Use
ex	x-height of font	Rare; font-specific fine alignment
cap	Capital letter height	Advanced typography alignment
ic	CJK ideographic character size	International text layouts
lh	Current element line-height	Vertical rhythm, text-related spacing
rlh	Root element line-height	Global vertical rhythm based on root line-height

■ **Real Use:** Beginners ke liye `rem`, `em`, aur `ch` sabse important hain. Advanced typography mein `lh`, `rlh`, `cap` useful ho sakte hain.

Percentage Units

Parent ya reference value ke relative size

DEFINITION

% unit relative hoti hai, lekin kis cheez ke relative hai yeh property par depend karta hai. Width mein parent width ke relative, font-size mein parent font-size ke relative, transform mein element ke khud ke size ke relative.

1. Width and Height Mein %

width: 50% ka matlab parent content width ka 50%. Agar parent 800px wide hai, child 400px wide hoga.

percentage width

```
.parent {  
  width: 800px;  
}  
  
.child {  
  width: 50%; /* 400px */  
}
```

PERCENTAGE WIDTH DEMO

25%



50%



75%



2. Height Percentage Ka Common Confusion

height: 100% tabhi properly work karta hai jab parent ki height explicitly defined ho. Agar parent ki height auto hai, browser ko 100% calculate karne ke liye fixed reference nahi milta.

● ● ● height percentage

```
/* problem */
.parent {
  height: auto;
}

.child {
  height: 100%; /* unclear reference */
}

/* better */
.parent {
  min-height: 100vh;
}
```

▪ 3. Margin and Padding Mein %

Horizontal aur vertical padding/margin percentage usually containing block ki inline size, yani width, ke relative calculate hoti hai. Isliye `padding-top: 50%` old responsive aspect-ratio trick mein use hota tha.

● ● ● old aspect ratio trick

```
.video-box {
  width: 100%;
  padding-top: 56.25%; /* 16:9 ratio */
}

/* modern better way */
.video-box {
  aspect-ratio: 16 / 9;
}
```

▪ 4. % Best Uses

Use Case	Example	Reason
Fluid images	max-width: 100%	Image parent se bahar overflow nahi karegi
Responsive sections	width: 90%	Screen ke according shrink/grow
Transform centering	translate(-50%, -50%)	Element ke own size ke relative move
Grid/Flex children	flex-basis: 50%	Columns parent space share karte hain

■ **Practical Pattern:** Common container pattern: `width: min(100% - 2rem, 1100px); margin-inline: auto;` . Isse mobile par spacing bhi milti hai aur desktop par max-width bhi.

Viewport Units

Screen/viewport ke size ke relative sizing

DEFINITION

Viewport units browser ke visible area ke relative calculate hoti hain. Full-screen hero sections, responsive typography, panels, dashboards, and mobile layouts mein yeh very useful hain.

1. vw and vh

Unit	Meaning	Example
vw	1% of viewport width	100vw = full viewport width
vh	1% of viewport height	100vh = full viewport height
vmin	1% of smaller side	Useful for square responsive sizing
vmax	1% of larger side	Large background/visual scaling

viewport example

```
.hero {  
  min-height: 100vh;  
  padding-inline: 5vw;  
}  
  
.hero-title {  
  font-size: 8vw;  
}
```

■ **Warning:** Pure `font-size: 8vw` mobile par too small aur desktop par too large ho sakta hai. Responsive font ke liye `clamp()` better hai.

2. Mobile Problem: 100vh

Mobile browsers mein address bar show/hide hota rehta hai. Old `100vh` kabhi visible area se bada ya chhota feel ho sakta hai. Isliye modern CSS mein small, large, dynamic viewport units aaye.

Unit	Meaning	Use
svh / svw	Small viewport height/width	Address bar visible state ke safe size ke liye
lvh / lvw	Large viewport height/width	Address bar hidden state ka larger size
dvh / dvw	Dynamic viewport height/width	Visible viewport change ke saath update hota hai

modern full height

```
.screen {  
  min-height: 100dvh;  
}  
  
.safe-screen {  
  min-height: 100svh;  
}
```

3. vi and vb - Logical Viewport Units

`vi` viewport inline size ka 1% hai, aur `vb` viewport block size ka 1% hai. English horizontal writing mode mein `vi` mostly `vw` jaisa aur `vb` mostly `vh` jaisa hota hai. Different writing modes mein yeh logical direction follow karte hain.

Viewport Units Best Practices

DO

Hero sections mein min-height use karo.
Mobile full screen ke liye 100dvh try karo.
Fluid spacing mein small vw values use karo.
Font size ko clamp ke saath control karo.

AVOID

Body width: 100vw without reason.
Fixed 100vh mobile pages.
Pure vw font-size without min/max.
Horizontal overflow create karna.

fr and Container Query Units

Advanced responsive layout ke powerful units

1. fr - Fraction Unit

fr CSS Grid ka special unit hai. Yeh available free space ka fraction represent karta hai. Agar grid columns **1fr 1fr** hain, available space 2 equal parts mein divide ho jata hai.

● ● ● grid fr unit

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  gap: 1rem;  
}
```

Yahan columns ka ratio 1:2:1 hoga. Middle column side columns se double width lega.

minmax() Ke Saath fr

Grid mein **minmax()** aur **fr** ka combination responsive cards ke liye excellent hota hai.

● ● ● responsive card grid

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 1rem;  
}
```

2. Container Query Units

Container units viewport ke relative nahi, balki nearest query container ke size ke relative hoti hain. Yeh component-based responsive design ke liye advanced and powerful feature hai.

```
.card-wrapper {  
  container-type: inline-size;  
}  
  
.card-title {  
  font-size: 8cqi;  
}
```

■ Container Units Table

Unit	Meaning	Similar To
cqw	1% of container width	vw, but for container
cqh	1% of container height	vh, but for container
cqi	1% of container inline size	logical width
cqb	1% of container block size	logical height
cqmin	1% of smaller container side	vmin
cqmax	1% of larger container side	vmax

■ **Advanced Tip:** Viewport units page-level responsiveness ke liye good hain. Container units component-level responsiveness ke liye better hain, especially cards, sidebars, widgets, and reusable components.

calc(), min(), max(), clamp()

Units ko combine karke smart responsive values banana

1. calc() - Calculation Karna

`calc()` different units ko combine karne deta hai. Example: full width minus fixed sidebar, ya 100% minus spacing.

calc examples

```
.main {  
  width: calc(100% - 280px);  
}  
  
.container {  
  width: calc(100% - 2rem);  
}
```

2. min() - Chhoti Value Choose Karna

`min()` given values mein se smallest value choose karta hai. Responsive container banane ke liye very useful.

min container

```
.container {  
  width: min(90%, 1100px);  
  margin-inline: auto;  
}
```

3. max() - Badi Value Choose Karna

`max()` given values mein se largest value choose karta hai. Minimum readable spacing ya safe touch target maintain karne mein useful.

max spacing

```
.section {  
  padding-top: max(2rem, 8vh);  
}
```

4. clamp() - Minimum, Preferred, Maximum

`clamp(min, preferred, max)` responsive design ka superstar hai. Yeh value ko minimum se chhota aur maximum se bada hone nahi deta, while preferred value viewport ke saath fluidly change ho sakti hai.

```
clamp(minimum, preferred, maximum)
```

fluid typography

```
h1 {  
  font-size: clamp(2rem, 6vw, 5rem);  
}  
  
.card {  
  padding: clamp(1rem, 3vw, 2.5rem);  
}
```

Math Functions Quick Table

Function	Meaning	Best Use
<code>calc()</code>	Calculate mixed units	100% - sidebar, spacing adjustments
<code>min()</code>	Choose smallest	Responsive max-width without separate max-width
<code>max()</code>	Choose largest	Minimum spacing/touch area
<code>clamp()</code>	Value between min and max	Fluid font-size, spacing, layout sizes

■ **Spacing Rule:** `clamp()` se responsive values professional feel karti hain, kyunki mobile par too small/too large problem control mein rehti hai.

Angle, Time, Resolution and Special Units

Sizing ke alawa CSS mein use hone wali important unit families

1. Angle Units

Angle units rotation, gradients, transforms, and animations mein use hoti hain.

Unit	Meaning	Example
deg	Degrees	rotate(45deg)
rad	Radians	rotate(3.14rad)
grad	Gradians	rotate(100grad)
turn	Full turns	rotate(0.5turn) = 180deg

● ● ● angle units

```
.badge {  
  transform: rotate(8deg);  
}  
  
.loader {  
  transform: rotate(1turn);  
}
```

2. Time Units

Time units transitions and animations mein use hoti hain.

Unit	Meaning	Example
s	Seconds	animation-duration: 2s
ms	Milliseconds	transition-duration: 250ms

time units

```
.button {  
  transition: background-color 200ms ease, transform 0.2s ease;  
}
```

3. Resolution Units

Resolution units media queries mein high-density screens detect karne ke liye use hoti hain.

Unit	Meaning	Use
dpi	Dots per inch	Print/screen resolution query
dpcm	Dots per centimeter	Resolution query
dppx	Dots per CSS pixel	Retina/high-DPI media query

resolution media query

```
@media (min-resolution: 2dppx) {  
  .logo {  
    background-image: url("logo@2x.png");  
  }  
}
```

4. Flex Fractions Are Not Units

`flex: 1` mein `1` unit nahi hai. Yeh flex-grow value hai. Grid ka `fr` unit hota hai, flex ka grow/shrink number unitless hota hai.

flex unitless grow

```
.item {  
  flex: 1; /* unitless flex grow shorthand */  
}
```

Real Projects Mein Units Kaise Choose Karein?

Professional responsive sizing strategy

■ Recommended Unit Strategy

CSS Property / Area	Recommended Units	Reason
Body font-size	rem / default 16px	User accessibility settings respect hoti hain
Headings	rem + clamp()	Responsive but controlled typography
Paragraph width	ch	Reading line length comfortable
Spacing	rem	Consistent design scale
Component padding	em or rem	em text ke saath scale, rem global scale
Layout width	%, min(), max-width	Fluid but controlled containers
Full screen section	dvh / svh / min-height	Mobile viewport behavior better
Grid columns	fr, minmax()	Available space smartly divide hota hai
Borders	px	Exact visual detail
Animations	ms / s	Time values clear and readable

■ Professional Responsive Template

```
html {  
  font-size: 16px;  
}  
  
.container {  
  width: min(100% - 2rem, 1120px);  
  margin-inline: auto;  
}  
  
.section {  
  padding-block: clamp(3rem, 8vw, 7rem);  
}  
  
h1 {  
  font-size: clamp(2.25rem, 6vw, 5rem);  
  line-height: 1.05;  
}  
  
p {  
  max-width: 65ch;  
  line-height: 1.7;  
}
```

■ Common Unit Mistakes

MISTAKE

Every width fixed px mein.
100vw body par use karke overflow lana.
100vh mobile hero without testing.
Huge vw font-size without clamp.
Nested em font sizes accidentally compound karna.
Height: 100% without parent height.

FIX

width + max-width combination use karo.
width: 100% usually safer hai.
min-height: 100dvh / 100svh try karo.
clamp(min, fluid, max) use karo.
Global scale ke liye rem use karo.
Parent height ya min-height define karo.

■ **Final Rule:** Unit choose karte time pehle question pucho: value fixed honi chahiye, parent ke relative, root font ke relative, viewport ke relative, ya container ke relative? Answer hi unit decide karega.

CSS Sizing Units - Complete Summary

All important units ek jagah revision ke liye

Unit	Type	Meaning	Use
px	Absolute	CSS pixel	Borders, icons, exact details
%	Relative	Reference depends on property	Fluid widths, transforms
em	Font relative	Current/parent font-size	Component padding, scalable parts
rem	Font relative	Root font-size	Font sizes, spacing scale
ch	Font relative	Width of o character	Readable text width
lh / rlh	Line relative	Current/root line-height	Vertical rhythm
vw / vh	Viewport	1% viewport width/height	Hero sections, fluid spacing
vmin / vmax	Viewport	Smaller/larger viewport side	Responsive squares, visuals
svh / lvh / dvh	Modern viewport	Small/large/dynamic viewport height	Mobile full-height layouts
fr	Grid	Fraction of available grid space	CSS Grid columns/rows
cqw / cqh / cqi / cqb	Container	1% of query container size	Component-based responsive design
deg / turn	Angle	Rotation angle	Transforms, gradients
s / ms	Time	Seconds / milliseconds	Animation and transition timing
dppx / dpi	Resolution	Pixel density	High-resolution media queries

■ Fast Decision Guide

Question	Choose
Exact small visual detail chahiye?	px
Root font-size ke saath scale chahiye?	rem
Component text ke saath spacing scale chahiye?	em
Parent width ke according size chahiye?	%
Viewport ke according size chahiye?	vw, vh, dvh
Container ke according component adapt chahiye?	cqi, cqw, cqmin
Grid space divide karna hai?	fr
Responsive min-max control chahiye?	clamp(), min(), max()

■ Best Practices - Yaad Rakhna

DO

Spacing system rem mein banao.
Paragraph max-width ch mein rakho.
Responsive typography ke liye clamp use karo.
Grid layouts mein fr + minmax use karo.
Mobile full screens mein dvh/svh test karo.

AVOID

Har layout fixed px mein banana.
Uncontrolled vw font sizes.
100vw se horizontal scroll create karna.
Height percentage bina parent height ke.
Random units without design logic.

> NEXT TOPICS TO COVER

Display Property

Flexbox

CSS Grid

Positioning

Responsive Design

Media Queries

Background Images

CSS Animations

Made with ❤️ for Web Development Course Learners
Shauray ke saath CSS seekhte raho 🚀 | Topic 06 - CSS Sizing Units